

NCERT Solutions For Class 12 Computer Science

Chapter 2 – File Handling in Python

Exercise Solutions: File Handling in Python

1. Differentiate between:

(a) text file and binary file

A text file contains human-readable characters, with each byte representing a character, and can be opened by any text editor. Its data is stored as ASCII or Unicode values. A binary file contains data as a stream of bytes representing images, audio, video, or executable files, which are not human readable and require specific programs to access. Any bit change in a binary file can corrupt the file.

(b) readline() and readlines()

The readline() method reads a single complete line from a file, ending with a newline character, and returns it as a string. The readlines() method reads all lines from a file and returns a list of strings, each string representing a line (including the newline character).

(c) write() and writelines()

The write() method writes a single string to a file and returns the number of characters written. The writelines() method writes a list (or other iterable) of strings to a file, without returning the number of characters written. Newline characters must be added manually to each string in the list for separate lines.

2. Write the use and syntax for the following methods:

(a) open()

Use: The open() method opens a file with a specified name and mode, returning a file object (file handle) for further operations.

Syntax: file_object = open(file_name, access_mode)

(b) read()

Use: The read() method reads data from the file. If an integer n is specified, it reads n bytes. Otherwise, reads the whole file.

Syntax: file_object.read(n)

(c) seek()

Use: The seek() method positions the file object at a specified byte position in the file.

Syntax: file_object.seek(offset [, reference_point])

(d) dump()

Use: The dump() method serializes (pickles) a Python object and writes it to a binary file.

Syntax: pickle.dump(data_object, file_object)

3. Write the file mode that will be used for opening the following files.

Also, write the Python statements to open the following files:

(a) a text file "example.txt" in both read and write mode

File Mode: 'r+'

Python Statement: fh = open("example.txt", "r+")

(b) a binary file "bfile.dat" in write mode

File Mode: 'wb'

Python Statement: fh = open("bfile.dat", "wb")

(c) a text file "try.txt" in append and read mode

File Mode: 'a+'

Python Statement: fh = open("try.txt", "a+")

(d) a binary file "btry.dat" in read only mode.

File Mode: 'rb'

Python Statement: fh = open("btry.dat", "rb")

4. Why is it advised to close a file after we are done with the read and write operations? What will happen if we do not close it? Will some error message be flashed?

It is advised to close a file after read and write operations to free up system resources (like memory) and to ensure that all unwritten or unsaved data is properly written (flushed) to the file. If a file is not closed, Python may still close it automatically when the file object is deleted or when the program ends, but this is not a good practice and could lead to data loss or resource leaks. Generally, no error message will be shown if not closed, but it is best practice to close files explicitly.

5. What is the difference between the following set of statements (a) and (b):

(a) P = open("practice.txt","r")

P.read(10)

This opens "practice.txt" in read mode and reads 10 bytes. However,

the file must be closed explicitly using `P.close()`, else it remains open until garbage collected or the program ends.

(b)

```
with open("practice.txt", "r") as P:  
    x = P.read()
```

This opens the file in read mode using the with clause and reads the entire content. The file is closed automatically after the block ends, even if an exception occurs, making it safer and more concise.

6. Write a command(s) to write the following lines to the text file named hello.txt. Assume that the file is opened in append mode.

“Welcome my class”

“It is a fun place”

“You will learn and play”

```
file = open("hello.txt", "a")  
lines = [  
    "Welcome my class\n",  
    "It is a fun place\n",  
    "You will learn and play"  
]  
file.writelines(lines)  
file.close()
```

7. Write a Python program to open the file hello.txt used in question no 6 in read mode to display its contents.

What will be the difference if the file was opened in write mode instead of append mode?

```
f = open("hello.txt", "r")
content = f.read()
print(content)
f.close()
```

If the file was opened in write mode instead of append mode, the existing contents would be overwritten and lost before new lines are written. Append mode adds new lines to the end, preserving previous contents.

8. Write a program to accept string/sentences from the user till the user enters “END”. Save the data in a text file and then display only those sentences which begin with an uppercase alphabet.

```
f = open("new.txt", "w")
while True:
    st = input("Enter next line:")
    if st == "END":
        break
    f.write(st + '\n')
f.close()
```

```
f = open("new.txt", "r")
for line in f:
    if line and line[0].isupper():
        print(line, end='')
f.close()
```

9. Define pickling in Python. Explain serialization and deserialization of Python object.

Pickling is the process of converting a Python object into a byte stream for storage in a binary file. Serialization is transforming data or an object in

memory to a stream of bytes (pickling), which can be saved or transmitted. Deserialization (unpickling) reverses this process, reconstructing the object from the byte stream.

10. Write a program to enter the following records in a binary file:

Item No	integer
Item_Name	string
Qty	integer
Price	float

Number of records to be entered should be accepted from the user. Read the file to display the records in the following format:

Item No:

Item Name:

Quantity:

Price per item:

Amount: (to be calculated as Price * Qty)

```
import pickle

with open("item.dat", 'wb') as itemfile:
    n = int(input("How many records to be entered? "))
    for i in range(n):
        ino = int(input("Enter item no: "))
        iname = input("Enter item name: ")
        qty = int(input("Enter quantity: "))
        price = float(input("Enter price: "))
        item = {"Item no": ino, "Item Name": iname, "Qty":
qty, "Price": price}
```

```
    pickle.dump(item, itemfile)
    print("Successfully written item data")
with open("item.dat", "rb") as file:
    try:
        while True:
            item = pickle.load(file)
            print("\nItem No:", item["Item no"])
            print("Item Name:", item["Item Name"])
            print("Quantity:", item["Qty"])
            print("Price per item:", item["Price"])
            print("Amount:", item["Qty"] * item["Price"])
    except EOFError:
        pass
```